

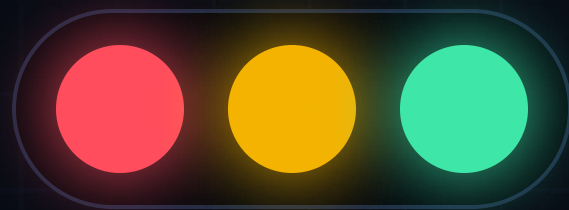
IBM BOB 2.0 HACKATHON

Every agent finished. The build didn't.

SIGNALBOX

Every Bob agent at once. Nothing unproven gets in.

The missing control layer for multi-agent development: railway interlocking for IBM Bob subagents.



THE PROBLEM

Every agent finished. The build didn't. And nobody can tell which change broke it.

They collide

Two agents edit the same file.
One silently overwrites the other.

They break each other

One renames a function the other two still call.

They cheat

A failing test gets "fixed" by rewriting the test.

What's left is a 4,000-line diff nobody can review and a morning of lost work. So teams stop trusting agents in parallel and go back to **one at a time**, and the work that needs parallel agents most stays slow.

THE INSIGHT

Railways solved "many trains, one network" 150 years ago.



Block

A section of track. One train at a time.

→ the files one subagent owns



Signal

No entry until the line ahead is clear.

→ a wave opens when earlier waves clear



Track circuit

Proves the block is empty before release.

→ four checks before any commit

This is **interlocking**. Signalbox brings it to AI agents.

HOW IT WORKS

Bob plans, Bob works in parallel, the signal box keeps it safe.

1 Scan

chainguard finds 72 legacy call sites and reads the import graph

2 Plan

6 blocks in 2 waves. Blocks in a wave never touch each other

3 Dispatch

A Bob dispatcher agent starts a subagent for every green block

4 Claim

Each subagent claims its block. Everyone else is locked out

5 Prove

Release runs the track circuit on an isolated copy

6 Commit

A clear block is committed alone and signed by its agent. Otherwise, fix or roll back

No block is released until four checks pass.

S

Scope

Only this block's files changed. Anything else is a **SPAD** (signal passed at danger).

C

Contract

No export another file still imports was removed or renamed.

L

Legacy scan

Zero old-library calls left in the block.

T

Isolated tests

Run on a git worktree with only cleared work plus this block, so agents can't cause or mask each other's failures.

THE SAFETY PROOF • ONE CLICK

3 agents at once. One goes rogue. It never reaches a commit.



Rogue agent

Edits a file outside its block **and** renames an export another file imports. Caught by scope **and** contract: 1 SPAD, 1 contract break.



Rolled back

Its block and the stray `index.js` restored from git. Nothing it did was committed.



Others untouched

2 blocks pass all four checks and commit alone, signed by their agents. Up to 3 agents at once.

All changes proven. Zero collisions. Ledger verified. Parallel agents are now safe.

Real recorded run, 10 SHA-256 chained events. Scripted drill agents, so the rogue is guaranteed; the same signal box Bob's subagents run under. Try it: bob-chainguard.vercel.app/?prove

IBM BOB DOES THE WORK

Every capability the challenge names, doing real work.

Full-repo context + document understanding

Bob reads the repo and the migration playbook before touching anything

Agent mode, multi-step orchestration

A dispatcher agent reads the signal box, starts subagents, handles faults, moves wave by wave

Subagents + parallel tasks

One subagent per block, a whole wave at once, on files guaranteed not to overlap

Self-correction

A red track circuit hands Bob the failing test. Bob fixes the code, not the test

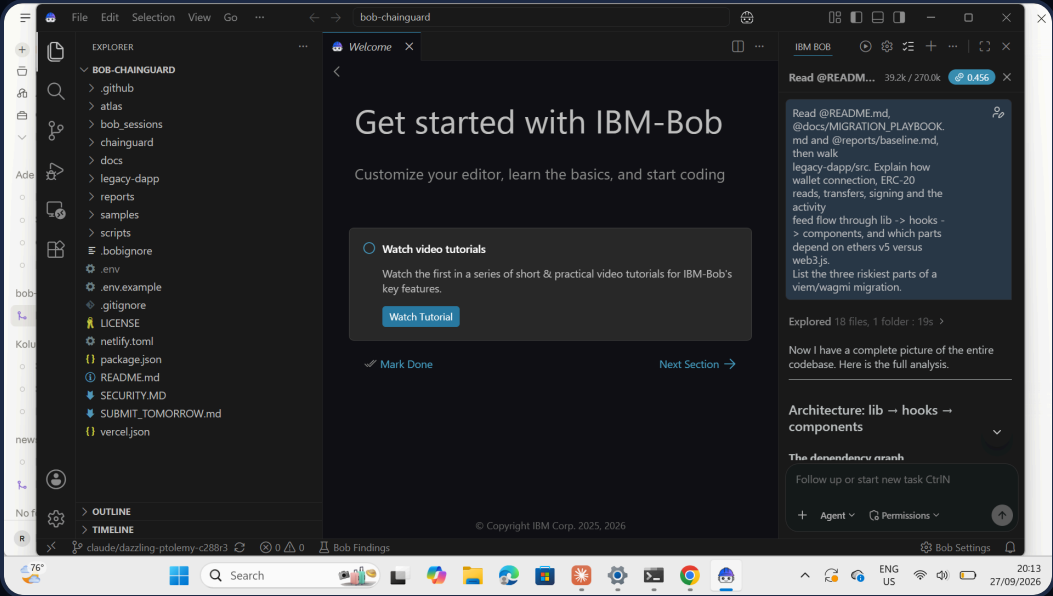
Code review

Bob reviews the full diff using the ledger of who changed what

SIGNALBOX

IBM BOB AT WORK

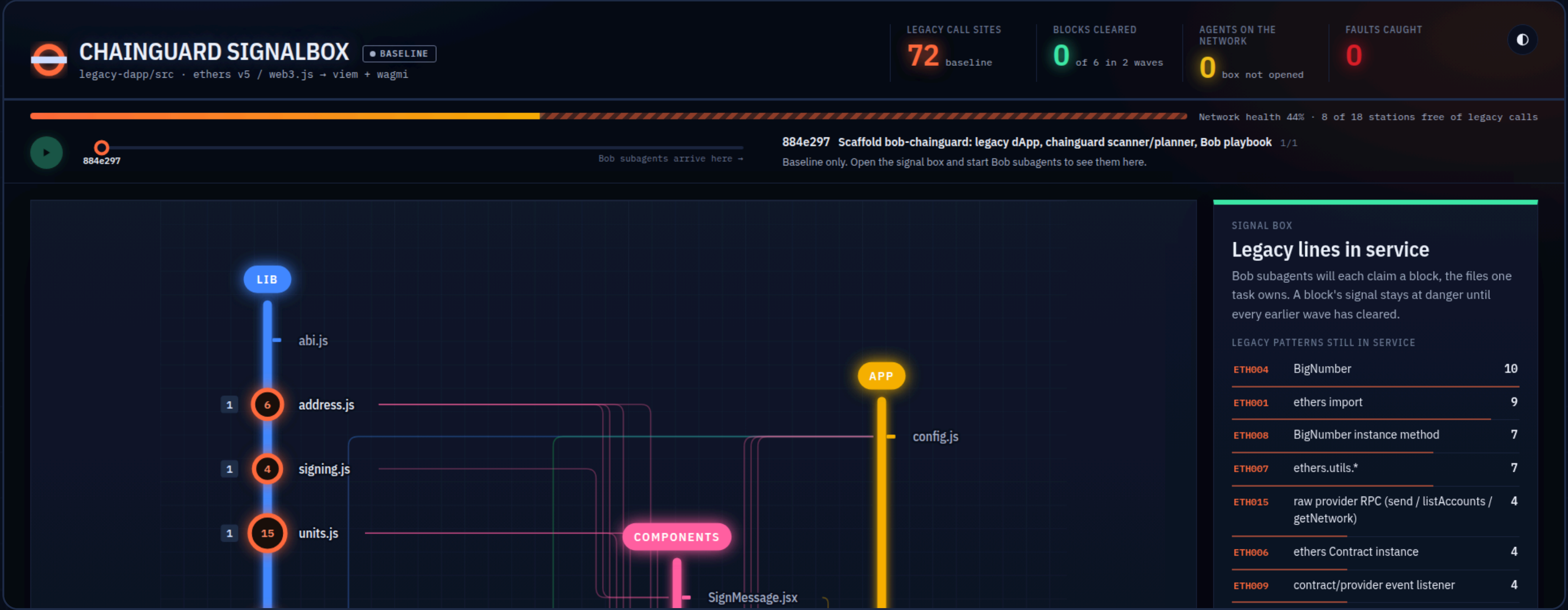
1 Bob session, straight from the run.



Onboarding: Bob reads the repo and the
playbook · ramadan

THE LIVE PANEL

Watch the agents work. Replay every decision.



Files are stations, folders are lines, imports are tunnels. Blocks light up as agents enter, faults flash red with the failing test, and the train graph shows the agents running in parallel.

A control tower, a rogue-agent proof and a desk that acts.



Control tower

One lane per Bob subagent, streamed from the ledger: the block it holds, its last move, and an amber **Held at signal** the moment interlocking refuses a claim.



Prove safety

One click: three agents enter at once, one goes **rogue** (strays out of its block, breaks an export). Caught before commit, rolled back, the other two commit. **Zero collisions. Ledger verified.**



Desk

"Start all safe wave 1" hands Bob the dispatch. "Simulate bad agent" runs the proof. "Why is this signal at danger?" answers from the ledger. Bob uses it too: **sb ask.**

RESULTS FROM THE REAL RUN

Real numbers from the ledger, not a slide.

72 → 0

legacy call sites

6/6

blocks cleared by Bob

6

Bob subagents, up to 3 at once

0

faults caught before commit

1 / 0

claims refused at signal / chaos
drills caught

4m 17s

Bob's run, first claim to last clear

Behavior tests: 22/22, never modified. The deployed panel replays this exact run.

BUILT TO BE TRUSTED

Safe even when an agent misbehaves. Useful beyond this demo.

Safety net

- Tamper-evident ledger: SHA-256 hash chain, audited against git in CI and re-verified in the browser, with a live tamper test
- Tests and the checker are protected: agents can't edit them
- Live SPAD alarm the second an unclaimed file changes, proven on demand by the chaos drill
- Pre-commit guard: no bypassing the signal box
- Black-box recorder restores work a rogue `git checkout` would wipe
- Expand → migrate → contract: shared code changes last, and only when unused

Any migration

- Rule packs: Web3 (ethers/web3.js → viem) built in
- Moment.js → date-fns pack included, 13 rules + playbook, switchable live in the panel on a sample app (42 call sites, 4 blocks) and proven end to end in CI
- Same interlocking for framework upgrades, API renames, library swaps
- Review each agent's exact diff, and see any file's blast radius
- One command after the run: `npm run finalize`



**Every Bob agent at once.
Nothing unproven gets in.**

The missing control layer for multi-agent development.

bob-chainguard.vercel.app · github.com/ramadan904/bob-chainguard